

vaishnavi badoge

Aim: Building a Simple Data Science Pipeline: Integrate all learned techniques in previous labs into a complete workflow.

Theory

1. What is a Data Science pipeline? Explain its key stages with examples.

Ans: A Data Science pipeline is a step-by-step process used to collect, process, analyze, and model data in order to solve a problem or make predictions. It helps organize the complete workflow of a data science project. The main stages of a data science pipeline include data collection, data cleaning, exploratory data analysis (EDA), feature engineering, model building, evaluation, and deployment. For example, in a student performance prediction system, data is first collected from student records, cleaned by handling missing values, analyzed using graphs and statistics, transformed into useful features, and then used to train a machine learning model that predicts student marks. This pipeline helps ensure accuracy, efficiency, and better decision-making.

2. What is Exploratory Data Analysis (EDA)? How do univariate and bivariate analyses help in understanding the dataset?

Ans: Exploratory Data Analysis (EDA) is the process of analyzing and visualizing data to understand its structure, patterns, relationships, and important characteristics before applying machine learning models. EDA helps identify missing values, outliers, trends, and distributions in the dataset. Univariate analysis studies a single variable at a time and helps understand its distribution, central tendency, and spread using methods like histograms, bar charts, and summary statistics. For example, analyzing only student marks to see average performance is univariate analysis. Bivariate analysis studies the relationship between two variables using scatter plots, correlation, or grouped charts. For example, analyzing the relationship between study hours and marks helps understand whether increased study time improves performance. Together, these analyses provide a better understanding of the dataset.

3. Differentiate between Label Encoding and OneHot Encoding. When should each be used?

Ans: Label Encoding and OneHot Encoding are techniques used to convert categorical data into numerical form so that machine learning algorithms can process it. Label Encoding assigns a unique numerical value to each category. For example, Red = 0, Blue = 1, Green = 2. It is mainly used when the categories have an order or ranking, such as low, medium, and high. OneHot Encoding creates separate binary columns for each category. For example, if a color column contains Red, Blue, and Green, three separate columns are created where only one column has value 1 and the others have value 0. OneHot Encoding is preferred when categories do not have any natural order because it prevents algorithms from assuming relationships between category numbers.

4. Why is feature scaling required? Compare normalization and standardization.

Ans: Feature scaling is required because machine learning algorithms perform better when all features are on a similar scale. If one feature has very large values and another has very small values, the algorithm may give more importance to the larger values, leading to inaccurate predictions. Feature scaling improves model performance, training speed, and accuracy. Normalization and standardization are two common scaling techniques. Normalization transforms data into a fixed range, usually between 0 and 1, using minimum and maximum values. It is useful when data does not follow a normal distribution. Standardization transforms data so that it has a mean of 0 and a standard deviation of 1. It is commonly used when data follows a normal distribution and is preferred in algorithms such as linear regression and logistic regression.

5. What is feature engineering? Explain how creating new features can improve model performance.

Ans: Feature engineering is the process of creating, selecting, or transforming variables in a dataset to improve the performance of machine learning models. It helps extract useful information from raw data and makes patterns easier for algorithms to learn. Creating new features can improve model accuracy by providing additional meaningful information. For example, instead of using separate day, month, and year columns, a new feature such as "total experience in years" can be created. In a student dataset, combining attendance percentage and study hours may create a feature representing overall learning effort. Good feature engineering can reduce noise, improve prediction quality, and make models more efficient and reliable.

```
import pandas as pd

# Create dataset
data = {
    "Name": ["ruchi", "bhu", "manuu"],
    "Marks": [80, 90, 75]
}

df = pd.DataFrame(data)

print(df)
```

	Name	Marks
0	ruchi	80
1	bhu	90
2	manuu	75

```
import pandas as pd

# 1. Prepare the data
data = {
    "Marks": [70, 80, 90, 60, 75]
}

# 2. Create the DataFrame
df = pd.DataFrame(data)

# 3. Perform EDA (Calculation)
average_marks = df["Marks"].mean()
max_marks = df["Marks"].max()

# 4. Display Results
print("Average Marks:", average_marks)
print("Maximum Marks:", max_marks)
```

```
Average Marks: 75.0
Maximum Marks: 90
```

```
from sklearn.preprocessing import LabelEncoder

# 1. Define your categorical data
color = ["pink", "Blue", "Green", "Red"]

# 2. Initialize the encoder
encoder = LabelEncoder()

# 3. Fit and transform the data
# This assigns 0 to 'Blue', 1 to 'Green', and 2 to 'Red' based on alphabetical order
result = encoder.fit_transform(color)

# 4. View results
print(result) # Output: [2 0 1 2]
```

```
[3 0 1 2]
```

```
import pandas as pd

# Create sample data
data = pd.DataFrame({ "Color": ["pink", "Red", "Blue", "Green"] })

# One-hot encoding with integer output (dtype=int)
result = pd.get_dummies(data["Color"], dtype=int)

print(result)
```

```
   Blue  Green  Red  pink
0     0     0   0     1
1     0     0   1     0
2     1     0   0     0
3     0     1   0     0
```

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 1. Define your data as a 2D array (required by scikit-learn)
data = np.array([[10], [20], [30], [40]])

# 2. Initialize the scaler
scaler = MinMaxScaler()

# 3. Fit to data and transform in one step
result = scaler.fit_transform(data)

# 4. View the normalized output
print(result)
```

```
[[0.        ]
 [0.33333333]
 [0.66666667]
 [1.        ]]
```

```
from sklearn.preprocessing import StandardScaler
import numpy as np
```

```
# Your data
data = np.array([[50], [20], [30], [40]])

# Initialize the scaler
scaler = StandardScaler()

# Use fit_transform() with parentheses and the data argument
result = scaler.fit_transform(data)

print(result)
```

```
[[ 1.34164079]
 [-1.34164079]
 [-0.4472136 ]
 [ 0.4472136 ]]
```

```
import pandas as pd

# Initial data
data = {
    "Maths": [70, 80, 50],
    "arts": [75, 85, 95]
}

df = pd.DataFrame(data)

# Create new feature: Total score
df["Total"] = df["Maths"] + df["arts"]

print(df)
```

	Maths	arts	Total
0	70	75	145
1	80	85	165
2	50	95	145